LANGCHAIN 마이그레이션 도우미 · 팀 모두를 위한 안내서

프로젝트 전체 쉽게 보기

최신 버전 · 2026-10-12

우리 팀이 무엇을, 왜, 어떤 순서로 만드는지 **주방 이야기**로 설명합니다. 1~3번 팀원도, 처음 보는 사람도 읽을 수 있습니다.

다음 문서: 02 검증·평가 파트 상세 (4번 담당의 할 일)

이것 세 줄만 기억해도 됩니다

1. AI는 **옛날 주방**만 기억해서, 도구가 이사 간 줄 모르고 옛날 레시피를 줍니다.
2. 우리 프로그램은 **새 주방 안내서를 보고** 레시피를 고친 뒤, **서랍을 직접 열어 보며** 도구가 있는지 확인합니다.
3. 나(4번)는 **요리 대회 심사위원**입니다. 같은 문제 30개로 "우리가 더 잘 고친다"를 숫자로 증명합니다.

1장

프로젝트 전체를 주방으로 이해하기

프로그램을 만드는 일은 요리와 비슷합니다. **LangChain**은 AI 프로그램을 만들 때 쓰는 **주방 도구 세트**입니다. 그리고 코드 맨 위에 쓰는 `import`는 **"어느 서랍에서 어떤 도구를 꺼내라"**는 뜻입니다.

```
# 코드로 쓰면
from langchain.chat_models import ChatOpenAI

# 주방 말로 바꾸면
"langchain 서랍장의 chat_models 칸에서 ChatOpenAI 도구를 꺼내라"
```

그런데 2025년에 LangChain 1.0이 나오면서 **주방 정리를 새로 했습니다.** 도구들이 다른 서랍으로 이사를 갔고, 일부는 창고로 치워졌습니다.

도구	옛날 위치	지금 위치
ChatOpenAI (AI 모델 연결)	`langchain.chat_models`	`langchain_openai` 서랍으로 이사
PromptTemplate (질문 틀)	`langchain.prompts`	`langchain_core` 서랍으로 이사
LLMChain (옛날 조립 방식)	`langchain.chains`	창고(`langchain_classic`)로 치워짐. 요즘은 `prompt | llm`으로 조립
initialize_agent (옛날 에이전트)	`langchain.agents`	없어짐. 요즘은 `create_agent` 사용

옛날 레시피대로 옛날 서랍을 열면 이렇게 됩니다. 이건 실제로 최신 LangChain(1.4.4)에서 실행한 결과입니다.

```
from langchain.chat_models import ChatOpenAI
ImportError: cannot import name 'ChatOpenAI'    ← "그 칸에 그런 도구 없어요"

from langchain.chains import LLMChain
ModuleNotFoundError: No module named 'langchain.chains'    ← "그런 칸 자체가 없어요"
```

문제: 옛날 주방만 기억하는 요리사

ChatGPT 같은 AI는 인터넷 글을 많이 읽고 배웠습니다. 그런데 인터넷에는 **옛날 주방 레시피가 훨씬 많습니다.** 그래서 AI는 옛날 서랍 위치를 알려 주고, **자기가 틀린 줄도 모릅니다.** 이걸 "버전 지체 환각"이라고 부릅니다.

해결: 안내서를 보고, 서랍을 직접 열어 보는 도우미

일반 AI

기억에 의존해서 레시피를 고칩니다. 서랍을 열어 보지 않으니 틀려도 모릅니다.

우리 프로그램

① **새 주방 안내서(공식 문서)**를 찾아보고 고칩니다.
② **서랍을 직접 열어 봅니다**(import 실행). 도구가 없으면 다시 찾아보고 고칩니다.

①을 **문서 근거**, ②를 **실행 근거**라고 부릅니다. 두 가지로 확인한다고 해서 **"이중 검증"**입니다. 이게 우리 프로젝트의 핵심입니다.

헷갈리기 쉬운 점

우리 프로그램은 **주방을 정리하지 않습니다.** 새 주방은 LangChain 회사가 이미 정리해 두었습니다. 우리 프로그램이 고치는 것은 **레시피(사용자의 코드)**입니다. "옛날 서랍 A에서 꺼내라"를 "새 서랍 B에서 꺼내라"로 고쳐 쓰고, B를 열어 보며 도구가 진짜 있는지 확인합니다.

2장

기본 설계: 프로그램은 어떤 순서로 일하나

기본 설계는 앞에서부터 보는 방법입니다. 사용자가 옛날 레시피(코드)를 넣으면, 프로그램 안에서 다섯 명의 직원이 차례로 일합니다.

① 담당 3번	② 담당 2번	③ 담당 3번	④ 담당 1번	⑤ 담당 3번
`analyzer`	`retriever`	`rewriter`	`verifier`	`report`
레시피를 읽고 "이 도구들, 이사 갔을 수도 있어" 표시	새 주방 안내서에서 도구의 새 위치 찾기	안내서를 보고 레시피 고쳐 쓰기	새 주방에서 서랍을 직접 열어 보기	고친 레시피 + 안내서 몇 쪽 + 몇 번 만에 성공

④에서 도구가 없으면 → 에러 메시지를 들고 ②로 돌아가 다시 찾습니다. (최대 3번)

이 되돌아가는 길을 **검증 루프**라고 합니다. 일반 AI에는 이 길이 없습니다. 그래서 틀려도 그냥 끝납니다.

팀원 역할도 이 그림으로 이해됩니다

1번은 직원들을 줄 세우고 연결합니다(그래프·백엔드, verifier). 2번은 안내서를 준비하고 찾아 줍니다(데이터·검색). 3번은 읽고, 고쳐 쓰고, 답을 씁니다(변환·화면). **4번(나)은 이 직원들이 일을 잘하는지 시험하고 점수를 매깁니다.**

3장

역설계: 발표 장면에서 거꾸로 내려가기

역설계는 뒤에서부터 보는 방법입니다. "마지막에 무엇을 보여줄까?"를 먼저 정하고, "그걸 하려면 뭐가 필요하지?"를 계속 물으며 내려갑니다.

🏆 **발표: "같은 문제 30개에서 우리가 일반 AI보다 더 많이 맞혔습니다"**

이 장면을 보여주려면?

↓ 필요한 것

6 그래프

누가 몇 % 맞혔는지 한눈에 → `make_charts.py`

↓ 그래프를 그리려면

5 성적표

문제마다 맞았는지 틀렸는지 적은 파일 → `run_baseline.py`가 만듦

↓ 성적표를 만들려면

4 채점 로봇

서랍을 직접 열어 보고 ⭕❌ 판정 → `verify.py`

↓ 채점하려면

3 채점 규칙

공정한 시합 규칙표 → `EVAL_RULES.md`

↓ 규칙대로 시험을 치려면

2 시험지 30문제

옛날 레시피 30개 → `testset.json`, 검사기 `check_testset.py`

↓ 시험지를 만들려면

1 **자료 조사**

무엇이 어디로 이사 갔는지 알기 → `데이터_출처_조사.md`

두 설계는 무엇이 다른가요?

	기본 설계 (2장)	역설계 (3장)
방향	앞 → 뒤	뒤 → 앞
보는 것	**프로그램**이 일하는 순서	**내(4번)**가 일하는 순서
질문	"코드를 넣으면 무슨 일이 일어나지?"	"발표에서 보여주려면 뭐가 필요하지?"
쓰는 때	프로그램을 설명할 때 (발표 5번 슬라이드)	내 할 일 순서를 정할 때

역설계로 정했기 때문에, 실제로 일하는 순서는 계단을 **아래에서 위로** 올라가는 순서(1 → 6)입니다. 1~4번 계단은 이미 초안이 완성되어 있습니다.

4장

용어 사전

LangChain	AI 프로그램을 만드는 주방 도구 세트
버전 (0.x, 1.x)	주방 정리 몇 번째 판인지. 1.0에서 크게 다시 정리됨
import	"어느 서랍에서 어떤 도구를 꺼내라"는 코드 한 줄
패키지	서랍장 하나. 예: `langchain_openai`, `langchain_core`
ImportError	"그 서랍에 그런 도구 없어요"
ModuleNotFoundError	"그런 서랍 자체가 없어요"
마이그레이션	옛날 레시피를 새 주방에 맞게 고치는 일
버전 지체 환각	AI가 옛날 주방만 기억해서 틀린 위치를 자신 있게 알려 주는 현상
RAG	AI가 답하기 전에 안내서(문서)를 찾아보게 하는 방법
공식 문서	새 주방 안내서. LangChain 회사가 직접 쓴 설명서
가상환경 (verify-env)	시험용으로 따로 만든 깨끗한 새 주방
verifier / 검증	서랍을 직접 열어 보고 도구가 있는지 확인하는 단계
검증 루프	도구가 없으면 안내서를 다시 찾아 고치는 되돌아가는 길 (최대 3번)
baseline	비교 기준. 여기서는 일반 AI의 성적
테스트셋	시험 문제지. 옛날 레시피 30개
판정 / 채점	합격(⭕)인지 불합격(❌)인지 정하는 것

%p	퍼센트끼리 뺀 값. 20% → 90%는 +70%p
langchain_classic	옛날 도구를 모아 둔 창고. 꺼내 쓸 수는 있지만 낡은 방식
langchain_community	"곧 문 닫습니다" 공지가 붙은 가게
JSON	컴퓨터가 읽기 쉽게 정리한 표 같은 파일. 시험지와 성적표가 이 모양

5장

확인 퀴즈

답을 누르면 바로 맞았는지 알려 줍니다. **0 / 3 맞힘**

Q1. 옛날 코드 from langchain.chat_models import ChatOpenAI 를 최신 LangChain에서 실행하면?

잘 실행된다

에러가 난다. ChatOpenAI가 langchain_openai 서랍으로 이사 갔기 때문

경고만 나오고 실행된다

> 해설: 1.0에서 OpenAI 관련 도구는 langchain_openai라는 새 서랍장으로 이사 갔습니다. 옛날 칸을 열면 ImportError가 납니다.

Q2. 우리 프로그램이 일반 AI와 다른 두 가지 근거는?

속도 근거 + 가격 근거

문서 근거 + 실행 근거

인기 근거 + 기억 근거

> 해설: 새 주방 안내서(공식 문서)를 보고 고치고(문서 근거), 서랍을 직접 열어 확인합니다(실행 근거).

Q3. verifier에서 도구가 없다고 나오면 어떻게 하나요?

그냥 끝낸다

에러 메시지를 들고 retriever로 돌아가 다시 찾는다 (최대 3번)

사용자에게 직접 고치라고 한다

> 해설: 이 되돌아가는 길이 검증 루프입니다. 일반 AI에는 이 길이 없습니다.

최신 버전 · 2026-10-12 · 같은 내용의 MD와 PDF는 저장소 `docs/guide` 폴더에 있습니다. 이전 버전: `docs/역순설계_쉽게_이해하기.md`